Python for HPC

SDSC Summer Institute 2026
Andrea Zonca
Friday, August 7 | 8:30 AM to 11:20 AM

SAN DIEGO SUPERCOMPUTER CENTER

UC San Diego
HALICIOĞLU SCHOOL OF DATA SCIENCE AND COMPUTING

What you will be able to do

- Speed up a numerical loop with Numba and time it fairly.
- Choose threads or processes based on the kind of work.
- Choose Dask chunks and run one calculation on two nodes.

The slides lead the session

- Follow the file or action shown here.
- Work in the notebook while this slide stays visible.
- Return here for the debrief and next step.
- Blue means stuck. Yellow means ready.

Today has one core path

- Setup and performance workflow.
- Numba, then threads and processes.
- Dask on one node, then two nodes.
- Recap and take-home material.

1 of 7 | Setup

- Time check: 8:30 AM.
- Goal: open Jupyter and verify the environment.
- We move on when most laptops show yellow.

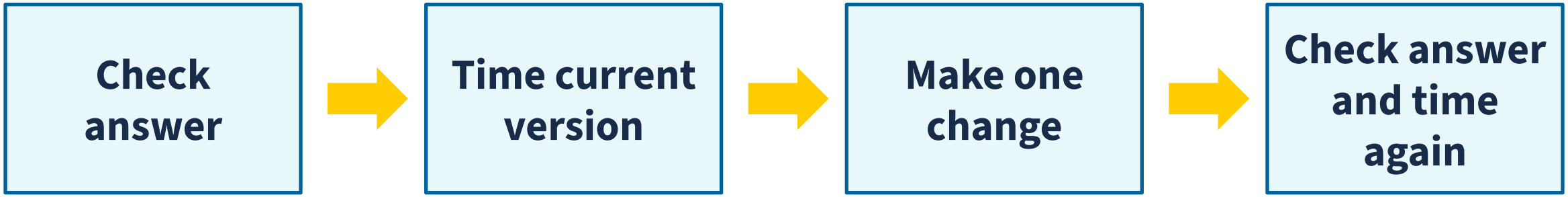
SETUP | Open the lesson

- In the repository, open folder 6.1_python_for_HPC.
- Run bash launch_galyleo_compute.sh. It creates or reuses the pythonhpc Conda environment.
- Keep one Jupyter terminal available.

SETUP | Run the import check

- In Jupyter, open the lesson README.md.
- Copy and run the four-package import cell.
- Yellow means imports work.
- Blue means a helper should stop by.

SETUP | Measure one change at a time



2 of 7 | Numba

- Time check: 8:42 AM.
- Goal: speed up one slow numerical loop and time it fairly.
- Core file: `3_numba/0_basics.ipynb`.

NUMBA | What @jit does

- Add @jit above a function to ask Numba to compile it.
- The first call compiles the function and returns an answer.
- Later calls reuse the compiled code.
- Check the answer, call once, then time later calls.

NUMBA | Open this notebook

- Open folder 3_numba.
- Open 0_basics.ipynb.
- Run through "Compare with NumPy."
- Predict each timing before you run it.
- Stop at "Your turn."

NUMBA | Pause and ask

- Did all three versions return the same value?
- Which timing includes compilation?
- Why do we call the function once before timing it?

NUMBA | Hands-on

- Complete sum_of_squares.
- Check the result against NumPy.
- Warm up, then time both versions.
- 12 minutes. Blue means help. Yellow means ready.

NUMBA | Debrief

- First call: compile and run.
- Later calls: reuse the compiled code.
- @jit can speed up supported numerical loops.
- Time check: 9:15 AM.

3 of 7 | Threads and processes

- Time check: 9:15 AM.
- Goal: predict the useful execution model.
- Notebook: threads versus processes.

THREADS OR PROCESSES | Mental model

- Threads share one process and its memory.
- The GIL is a CPython rule: one thread runs Python code at a time.
- Processes can run Python code on separate CPU cores.
- Starting processes and sending data takes time and memory.

THREADS OR PROCESSES | Open this notebook

- Open folder 4_threads_vs_processes.
- Open threads_vs_processes.ipynb.
- Read the mental model.
- Predict both comparisons before running them.
- Stop at "Your turn."

THREADS OR PROCESSES | Pause and ask

- Which workload spends its time calculating in Python?
- Which workload mostly waits?
- What data would be expensive to send to a process?

THREADS OR PROCESSES | Hands-on

- Run threads and processes with four workers.
- Explain each result with the mental model.
- Discuss one surprise with a neighbor.
- 12 minutes. Blue means help. Yellow means ready.

THREADS OR PROCESSES | Debrief

- Python calculations often favor processes.
- Waiting work often favors threads.
- Shared data can change the tradeoff.
- Time check: 9:40 AM.

4 of 7 | Break

- Time check: 9:40 AM.
- Break ends at 9:50 AM.
- The Dask section starts after the break.

5 of 7 | Dask tasks and chunks

- Time check: 9:50 AM.
- Goal: build a graph, then choose useful chunks.
- Notebooks: delayed, then multicore array.

DASK | Build a plan before running

- A task is one piece of work.
- A task graph is a plan showing tasks and their order.
- The scheduler assigns ready tasks to workers.
- compute() starts the work and returns the result.

DASK | Open the delayed notebook

- Open folder 5_dask.
- Open 1_delayed.ipynb.
- Run through "Delay calls, then compute once."
- Inspect what exists before compute().
- Stop at "Your turn."

DASK | Final summary and debrief

- Put the final summary inside the task graph.
- Compute only the final summary.
- 5 minutes. Blue means help. Yellow means ready.
- Then explain what compute() changed.

DASK | Chunks are units of work

- Too small: Dask manages too many tiny tasks.
- Too large: a chunk may not fit or share work evenly.
- Useful chunks fit in memory and keep workers busy.
- Inspect chunks before compute().

DASK | Open the array notebook

- In folder 5_dask, open 2_multicore_array.ipynb.
- Run through "Choose chunks."
- Compare the NumPy result with the Dask plan.
- Stop at "Your turn."

DASK | Chunk exercise

- Try chunk sides 250, 1000, and 3000.
- Keep the mathematical expression unchanged.
- Compare task count, memory, and timing.
- 10 minutes. Blue means help. Yellow means ready.

DASK | Debrief

- A graph is a plan, not a result.
- Chunks control scheduling and memory.
- NumPy can still win for small in-memory work.
- Time check: 10:20 AM.

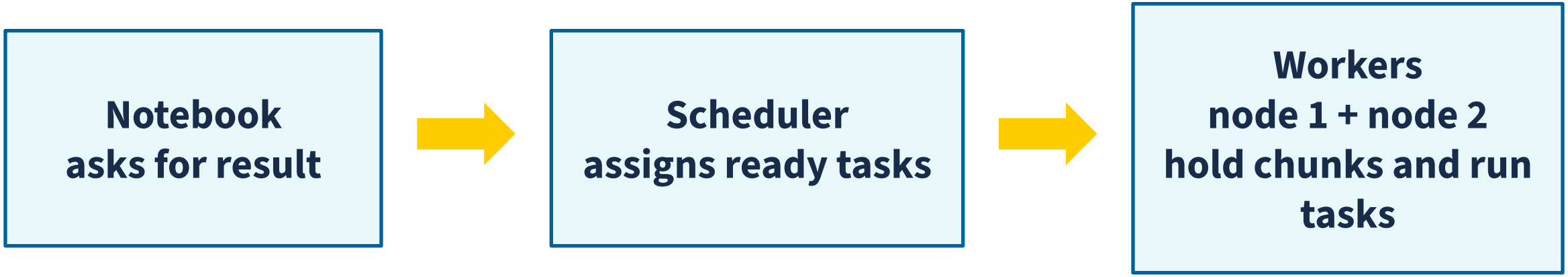
CLUSTER SETUP | 10 minutes

- In both terminals, cd 6.1_python_for_HPC.
- In a Jupyter terminal, run bash dask_slurm/launch_scheduler.sh.
- In a login terminal, run sbatch dask_slurm/dask_workers.slrm.
- Save the job number. Wait for two workers.

6 of 7 | Multi-node capstone

- Time check: 10:30 AM.
- Goal: run one Dask expression on two worker nodes.
- Notebook: multi-node distributed array.

CAPSTONE | Three roles



The array expression does not change.

CAPSTONE | Open this notebook

- Open folder 5_dask.
- Open 4_multinode_distributed_array.ipynb.
- Run "Start the cluster."
- Open the Dask dashboard and confirm two worker hosts.
- Stop before "Build an array across the workers."

CAPSTONE | Run and verify

- Build and calculate the array across both workers.
- Read worker hosts, threads, and result.
- Confirm the expected value.
- 18 minutes. Blue means help. Yellow means ready.

CAPSTONE | Debrief and clean up

- What changed when we added nodes?
- What stayed the same in the Python expression?
- Cancel the worker job and stop the scheduler.
- Time check: 11:02 AM.

7 of 7 | Recap

- Time check: 11:02 AM.
- Goal: choose the smallest useful layer.
- Final questions end at 11:20 AM.

RECAP | Decision map

- Slow numerical loop: Numba.
- Waiting tasks: threads or delayed.
- Python calculations: processes.
- Chunked or multi-node array: Dask.

RECAP | What you take home

- Core notebooks and optional practice notebooks.
- Production SI26 SLURM and Galyleo templates.
- A debug-queue validation workflow.
- Time check: 11:20 AM. Questions?

OPTIONAL | AI-assisted workflow

- Use only if recap and questions end early.
- Goal: test whether a suggestion is correct and useful.
- Otherwise, this section is take-home material.

AI | A reviewable loop

- State the environment, limits, and correct result.
- Ask for one small change and an explanation.
- Inspect commands and resource requests.
- Test correctness, then performance.

AI | Open this page

- Open folder 2_ai_code_assist.
- Open README.md.
- Read "A useful HPC prompt."
- Choose one function from the Numba notebook.
- Stop at "Hands-on: ask, inspect, test."

AI | Ask, inspect, test

- Ask for one Numba optimization.
- Identify every package and resource assumption.
- Run the answer check and time the result twice.
- 8 minutes. Accept, revise, or reject.

AI | Debrief

- Plausible output is not evidence.
- Generated SLURM needs a line-by-line resource review.
- Keep only a verified improvement.
- Optional section. Stop when needed.